

CHAPITRE V – LES FICHIERS SEQUENTIELS

1. Introduction

Toutes les structures de données que nous avons utilisées jusqu'à maintenant étaient stockées en mémoire, c'est-à-dire dans une zone volatile de la machine. En mémoire, la durée de vie d'une variable est égale au temps d'exécution du programme. Nous comprenons facilement que cette solution n'est pas envisageable pour l'informatisation d'une entreprise par exemple. Les données concernant un employé, un produit, un fournisseur ou un contrat doivent être mémorisées plus longtemps que la durée du programme. Les valeurs traitées doivent être recopiées sur un support non volatile (disque dur, CD ROM...).

Mais quelle structure utiliser pour stocker ce type de données ?

Les fichiers répondent bien à ce besoin. Ils servent à stocker des informations de manière permanente, entre deux exécutions d'un programme.

2. Notion de fichier

Un FICHIER (anglais : file) est un REGROUPEMENT LOGIQUE DE DONNEES MEMORISEES SUR UN SUPPORT PERMANENT (disque dur, par exemple) afin de permettre une réutilisation ultérieure des informations qu'il contient.

3. Organisation des fichiers

a. Fichiers structurés (fichiers texte)

Les fichiers structurés sont composés d'un ensemble de données élémentaires identifiables de manière précise au sein du fichier, et qui sont répétés pour former ce fichier. On parle alors d'**enregistrement** ou article ou enregistrement logique.

Ce type de fichier est couramment utilisé dès lors que l'on doit stocker des informations pouvant être assimilées à une base de données. Par exemple, si le fichier est destiné à mémoriser les coordonnées d'un certain nombre de personnes. Pour chacune, il faudra noter le nom, le prénom, le numéro de téléphone et l'e-mail. Dans ce cas, il peut paraître plus simple de stocker une personne par ligne du fichier (par enregistrement). Dit autrement, quand on prendra une ligne, on sera sûr qu'elle contient les informations concernant une personne, et uniquement cela. Cette catégorie est aussi appelée fichier texte. Un fichier texte est constitué par une collection d'enregistrements : répétition de la structure ligne.

b. Fichiers non structurés (fichiers binaires)

Le second type de fichier rassemble les fichiers qui ne possèdent pas de structure d'enregistrement. Les octets, quels qu'ils soient, sont écrits à la queue leu leu. Ces fichiers sont appelés des fichiers binaires.

➔ Exemple : Tous les fichiers qui ne codent pas une base de données sont obligatoirement des fichiers binaires tels que : un fichier son, une image, un programme exécutable...

4. Accès aux fichiers

Un autre critère de distinction entre les fichiers consiste en le type d'accès. On distingue principalement l'accès séquentiels et l'accès direct ou aléatoire.

Accès séquentiel : Un fichier séquentiel permet uniquement d'accéder aux données dans l'ordre de leur écriture : On ne peut accéder qu'à la donnée suivant celle qu'on vient de lire. On ne peut donc accéder à une information qu'en ayant au préalable examiné celle qui la précède. Dans le cas d'un fichier texte, cela signifie qu'on lit le fichier ligne par ligne (enregistrement par enregistrement).

Accès direct (ou aléatoire) : on peut accéder directement à l'enregistrement de son choix, en précisant le numéro de cet enregistrement. Mais cela veut souvent dire une gestion fastidieuse des déplacements dans le fichier. Un fichier à accès direct correspond à un tableau en mémoire : toutes ses composantes ont la même taille, on peut donc accéder directement à la Nième. Comme pour un tableau, les insertions et suppressions nécessitent des décalages des composantes suivantes, ce qui est en général trop long pour être envisageable.

Dans ce qui suit, nous nous intéresserons aux fichiers séquentiels.

5. Déclaration

Définition de type

Fil = fichier de reel (le type des éléments du fichier est reel)

Déclaration de variables

F : Fil ou F : fichier de reel

X1	X2	X3	...	Xn	EOF ou FDF
----	----	----	-----	----	------------

X1...Xn sont des variables ayant le même type (ou un type compatible) avec le type figurant dans la déclaration du fichier.

6. Instructions

a. Ouverture d'un fichier

Si l'on veut travailler sur un fichier, la première chose à faire c'est de l'**ouvrir**. L'instruction d'ouverture doit indiquer le mode d'ouverture du fichier. La forme générale de l'instruction est la suivante : **Ouvrir (variable fichier, mode d'ouverture)**

- Si on ouvre un fichier en mode **lecture**, on pourra uniquement récupérer les informations qu'il contient, sans les modifier.

Exp : ouvrir (F, lecture)

EOF (End Of File) ou FDF (Fin De Fichier) est une variable booléenne :

- = faux s'il existe encore des enregistrements et
- = vrai si la fin du fichier est atteinte.
- Si on ouvre un fichier en mode **écriture**, on pourra y écrire des informations ; mais les informations précédentes, si elles existent, seront **intégralement écrasées**.
- Si on ouvre un fichier en mode **ajout**, on pourra y ajouter des enregistrements sans écraser les informations déjà existantes. Par contre, ces informations ne pourront pas être lues, ni modifiées.

b. Lecture depuis un fichier

L'instruction de lecture d'un fichier récupère donc dans la variable spécifiée l'enregistrement suivant dans le fichier. « Suivant » par rapport au dernier enregistrement lu. C'est pour cela que le fichier est dit séquentiel.

Tête de lecture est actuellement dans cette position par exemple, donc on va lire X4

X1	X2	X3	...	Xn	EOF ou FDF
----	----	----	-----	----	------------

X1	X2	X3	...	Xn	EOF ou FDF
----	----	----	-----	----	------------

Lire (F,x) $\Leftrightarrow x \leftarrow F$

Exemple :

X1	X2	X3	...	Xn	EOF ou FDF
----	----	----	-----	----	------------

Ouvrir (F, Lecture)

Tant que non FDF(F) faire

Lire (F,x)

Ecrire(x)

FTQ

➔ Va afficher X1, X2, X3, etc

c. Ecriture dans un fichier

Ouvrir (F, Ecriture)

Ecrire (F,x) $\Leftrightarrow$ x est écrite dans le fichier

Exemple:

Ecrire un algorithme qui permet de saisir une suite de 10 éléments et de les écrire dans un fichier.

Algorithme Ecriture ()

Var

x,i : entier

F : Fichier

Debut

Ouvrir (F, Ecriture)

Pour i de 1 à 10 Faire

 Lire(x)

 Ecrire (F,x)

Fpour

Fin

d. Fermeture d'un fichier

La dernière instruction du programme est la fermeture du fichier : Fermer(F)

Exercice 1:

1. Ecrire une procédure permettant de créer et remplir le fichier (de type Fi_employés) qui contient des informations sur les employés d'une entreprise (matricule, nom, prénom, grade, salaire).

Types

Employé = Struct

 matricule : Entier

 nom : Chaîne[30]

 prénom : Chaîne[30]

 grade : Chaîne[10]

 salaire : Entier

FinStruct

Fi_employes = Fichier de Employe

Procédure Création(Var fe : Fi_employes)

Variables

 em : Employé (*variable tampon*)

Début

 Ouvrir(fe,E)

 Ecrire(« matricule de l'étudiant : »)

 Lire(em.matricule)

TantQue (em.matricule $\neq$ 0) **Faire**

 Ecrire(« Donner le nom de l'employé : »)

 Lire(em.nom)

```

    Ecrire (« Donner le prénom de l'employé : »)
    Lire (em.prénom)
    Ecrire (« Donner le grade de l'employé : »)
    Lire (em.grade)
    Ecrire (« Donner le salaire de l'employé : »)
    Lire (em.salaire)
    Ecrire (« matricule de l'employé : »)
    Lire (em.matricule)

```

```

    Ecrire (fe, em)

```

```

FinTQ

```

```

    Fermer (fe)

```

```

Fin

```

2. Ecrire une procédure permettant d'afficher la liste des employés à partir du fichier de type Fi_employés.

```

Procédure Consultation (Var fe : Fi_employés)

```

```

Variables

```

```

    em : Employe

```

```

Début

```

```

    Ouvrir (fe, L)

```

```

    TantQue (NON (FDF (fe))) Faire

```

```

        Lire (fe, em)

```

```

        Ecrire (em.matricule, em.nom, em.prénom, em.grade, em.salaire)

```

```

    FinTQ

```

```

    Fermer (fe)

```

```

Fin

```

3. Ecrire une fonction permettant d'afficher la liste des employés dont le salaire est compris entre 500 et 700 DT.

```

Procédure Consultationbis (Var fe : Fi_employes)

```

```

Variables

```

```

    em : Employe

```

```

Début

```

```

    Ouvrir (fe, L)

```

```

    TantQue (NON (FDF (fe))) Faire

```

```

        Lire (fe, em)

```

```

        Si (em.salaire >= 500 ET em.salaire <= 700) alors

```

```

            Ecrire (em.matricule, em.nom, em.prénom, em.grade, em.salaire)

```

```

        )

```

```

        FinSi

```

```

    FinTQ

```

```

    Fermer (fe)

```

```

Fin

```

4. Copier les salaires des employés dans un tableau T_Salaire

```

Type

```

```

    Tab : Tableau[1..50] : Entier

```

```

Procédure CopieSalaires (Var fe : Fi_employés, T_Salaire : Tab)

```

```

Variables

```

```

    em : Employé

```

```

    i : Entier

```

```

Début

```

```

    i <- 1

```

```

    Ouvrir (fe, L)

```

```

    TantQue (NON (FDF (fe))) Faire

```

```
    Lire(fe, em)
    T_Salaire[i] <- em.salaire
    i <- i+1
  FinTQ
Fermer(fe)
Fin
```